19. Host Firewalls: nftables, iptables, ufw

19. Host Firewalls: nftables, iptables, ufw

Phase: Phase 8 — Firewalls, NAT, and policy
Estimated time: 2 days
Slug: `19-firewall-linux`

Learning objectives

- List rules and policies
- Allow least ports
- Test with care to avoid lockout
- Prefer high-level tools when enough

Prerequisites

Complete prior lessons. Prefer a Linux VM for `ip`/`ss`/`nft` examples; macOS works for many `curl`/`dig` drills. Type every command yourself and keep a lab notes file.

Why this matters

Fresh VM: curl works locally, remote times out. Host firewall default-drop, wrong zone, or forgotten allow for the service port — `nft`/`iptables`/`ufw` show policy, not vibes.

Core concepts

What you must be able to explain:

- List rules and policies
- Allow least ports
- Test with care to avoid lockout
- Prefer high-level tools when enough

Mental model for `19-firewall-linux` (exercise kind `fw_default`):

- Prefer **measurable** checks (exit codes, ICMP replies, DNS rcode, HTTP status, socket state) over vibes.
- Separate **reachability** (can packets flow?) from **correctness** (right service, cert, route, identity).
- Ask: *which layer's tool falsifies my current hypothesis?* Then run that tool.
- Record the exact command that proved the root cause — that becomes your playbook snippet.

Worked commands

Run these on a lab host. Linux VM preferred for `ip`/`ss`/`tcpdump`/`nft`; macOS is fine for many `curl`/`dig`/`ping` drills. Your addresses and timings will differ — match the *shape* of the output and the conclusion you draw.

Example 1

```
sudo nft list ruleset 2>/dev/null | head -40 || sudo iptables -L -n -v | head -40
```

Sample output:

```
table inet filter {
  chain input { type filter hook input priority filter; policy drop;
```

Interpretation: Default **drop/deny** on inbound is the safe posture. Counters (iptables -v) show which rule matches.

Example 2

```
ss -tlnp | head
curl -sS -o /dev/null -w '%{http_code}\n' --connect-timeout 2 http://127.0.0.1:9
|| echo 'timeout/fail expected'
```

Sample output:

```
LISTEN ...
timeout/fail expected
```

Interpretation: Listeners vs policy: a service may listen yet still be unreachable remotely if SG/NACL/nft drops mid-path.

Example 3

```
ping -c 2 -W 1 10.255.255.254 || true
# compare to: nc -vz -w 2 host 443
```

Sample output:

```
100% packet loss
```

Interpretation: Silent drops often look like timeouts, not TCP RST. That distinction drives ‘firewall vs app refused’ decisions.

Mini-lab

1. Re-type every worked command for **Host Firewalls: nftables, iptables, ufw** and paste outputs into lab notes.
2. Introduce one controlled failure in the lab (wrong DNS, firewall drop, bad route) and recover using only this lesson’s tools.
3. Write a 5-line playbook: symptom → first command → expected healthy output.
4. Complete the exercise stubs; then explain each function’s return value without looking.

Failure modes

Symptom	Likely cause	Next check
Locking yourself out	See common mistakes	Re-run lesson commands
Disabling firewall instead of opening…	See common mistakes	Re-run lesson commands
Forgetting IPv6 (`ip6tables`/`ufw`)	See common mistakes	Re-run lesson commands
Timeout with no RST	Silent drop / blackhole	tcpdump + compare path

Study tip for Host Firewalls: nftables, iptables, ufw

Before each new command, write a one-line hypothesis. If the result falsifies it, update the hypothesis instead of stacking random tools. Note equivalents across OS (`ip` ↔ `ifconfig`/`route`, `ss` ↔ `netstat`). Re-run two commands from this lesson tomorrow from memory and explain the sample outputs out loud.

Evidence habit

When pages are down, prefer evidence (`ip route get`, resolver answer, packet capture, listener list) over intuition. Document **command → output → conclusion** in a lab notebook so the same path is reusable on-call. Change one variable at a time; write a one-line hypothesis before each new tool.

Common mistakes

- Locking yourself out
- Disabling firewall instead of opening one port
- Forgetting IPv6 (`ip6tables`/`ufw`)

Exercises

Work in `exercises/19-firewall-linux/`. Read that folder's `README.md` before coding.

```
cd exercises/19-firewall-linux
chmod +x test.sh exercise.sh 2>/dev/null || true
./test.sh
```

What you will build

Edit `exercise.sh` and implement `safe_default_inbound`, `linux_fw_modern`, `ubuntu_fw_frontend`. Each function is a tiny retrieval drill for **Host Firewalls: nftables, iptables, ufw** — return the exact tokens/flags/labels from the lesson. Keep the comment above each function describing when you would use that fact on-call. Run `./test.sh` until all 3 assertions pass.

Study loop

- **A (Drill):** make tests pass
- **B (Apply):** run 2–3 real commands from this lesson in your lab and note outputs
- **C (Challenge):** rewrite answers from memory next day

Do not open `solutions/19-firewall-linux/` until after an honest attempt.

Self-check

Answer without scrolling up:

1. ufw allow SSH command idea?
2. Why console access?
3. nft list command?

Further reading

- https://wiki.nftables.org/